Travel Agency Management

Project Report - C++ Guided Project (CPP44)

Enrollment No.:	IU2541230068
Student Name:	Vijay Adesara
Semester:	3
Subject:	C++ Guided Project

1. Introduction

Travel agencies handle a large amount of information every day: customer details, tour packages, seat availability, bookings, payments and cancellations. When this data is maintained manually on paper or in scattered spreadsheets, it leads to duplicate entries, overbooking, calculation errors and slow service.

Travel Agency Management is a console-based application developed in C++ that automates these day-to-day operations. The system allows an agency to maintain a catalogue of domestic and international tour packages, register customers, book seats, accept payments, cancel bookings with a refund, and generate summary reports. All records are stored in text files so that data is retained after the program is closed.

The project is built entirely using Object-Oriented Programming concepts - classes, inheritance, abstraction, polymorphism, operator overloading, exception handling and file handling - and uses the C++ Standard Template Library (STL) for managing collections of objects.

2. Problem Statement

Small and medium travel agencies often rely on manual registers to record customers and bookings. This approach has the following problems:

- Searching for a package or a past booking is slow and error-prone.
- Seat availability is not updated in real time, which can cause overbooking.
- Price calculation (taxes, visa and insurance charges) is done by hand and may be incorrect.
- Cancellations and refunds are difficult to track.
- Data may be lost or damaged, and there is no quick way to prepare reports.

There is a need for a simple, reliable software system that stores travel data permanently, validates user input, calculates prices automatically for different package types, and keeps seat counts consistent. This project aims to provide such a system using C++ and OOP principles.

3. Objectives

- To design and implement a menu-driven Travel Agency Management System in C++.
- To model real-world entities (customer, package, booking, payment) as classes with clear responsibilities.
- To apply inheritance and runtime polymorphism for different package types (Domestic / International).

- To prevent overbooking by maintaining and validating seat availability.
- To validate inputs and handle errors using C++ exception handling.
- To store and retrieve all data permanently using file handling.
- To document the design using UML diagrams (use case, class and sequence diagrams).
- To test the application with a defined set of test cases and record the results.

4. Functional Requirements

ID	Requirement
FR1	The system shall allow the admin to add, update, view and delete tour packages (domestic or international).
FR2	The system shall register new customers with a unique customer ID and validate name, phone and e-mail.
FR3	The system shall search packages by destination (case-insensitive, partial match) and list all available packages.
FR4	The system shall book a package for a registered customer for a given number of travellers, and shall reject the booking if seats are insufficient.
FR5	The system shall calculate the total amount automatically depending on the package type (taxes, visa and insurance).
FR6	The system shall accept payment for a booking using Cash, Card or UPI and update the payment status.
FR7	The system shall cancel a booking, restore the seats to the package and refund 80% of the paid amount.
FR8	The system shall display all bookings of a customer and all bookings in the agency.
FR9	The system shall generate a summary report (total bookings, cancellations, revenue).
FR10	The system shall automatically load data from files at start-up and save data to files after every change.

5. System / Project Flow

The program starts by loading all data files through the *FileManager*. If a file does not exist, an empty file is created. The *TravelAgency* object then displays the main menu inside a loop until the user chooses to exit. Every operation that modifies data saves the files immediately.

1. Start program and call `FileManager::loadAll()`.
2. Display the main menu (Package Management, Customer Management, Booking, Payment, Reports, Exit).
3. Read the user's choice and validate it.
4. Call the corresponding *TravelAgency* method (for example `createBooking()`).
5. Inside the method, validate inputs; if invalid, a *TravelException* is thrown and caught in the menu loop, which shows the error message.
6. On success, update the in-memory vectors and call `FileManager::saveAll()`.
7. Return to the main menu. When the user selects Exit, save data and terminate.

Menu Option	Action
1. Package Management	Add / Update / Delete / View / Search packages
2. Customer Management	Register customer, view customers
3. Booking	Book package, cancel booking, view bookings
4. Payment	Make payment for a booking
5. Reports	Revenue and booking summary
0. Exit	Save data and quit

6. Class Descriptions

Class	Data Members	Member Functions	Responsibility
Person (abstract)	id, name, phone, email (protected)	displayInfo() = 0, getId(), getName(), getPhone(), virtual ~Person()	Base class for all people in the system; stores common identity data.
Customer	address, loyaltyPoints (private)	displayInfo(), addPoints(), toFileString(), fromFileString()	Represents a registered customer; derived from Person.
Package (abstract)	packageId, destination, days, basePrice, seatsAvailable (protected)	calculatePrice(n) = 0, displayDetails(), bookSeats(n), releaseSeats(n), operator==, toFileString()	Base class for tour packages; manages seat availability and defines the pricing interface.
DomesticPackage	gstRate (0.05)	calculatePrice(n), displayDetails()	Price = basePrice x travellers + 5% GST.
InternationalPackage	visaFee, insuranceRate	calculatePrice(n), displayDetails()	Price = (basePrice + visaFee + insurance) x travellers + 5% GST.
Booking	bookingId, customerId, packageId, travellers, totalAmount, status	cancel(), display(), getters, toFileString(), operator<<	Stores one reservation and its status (Booked / Cancelled).
Payment	paymentId, bookingId, amount, mode, status	process(), refund(rate), toFileString()	Records payments and refunds linked to a booking.
TravelAgency	vector<Customer>, vector<Package*>, vector<Booking>, vector<Payment>	addCustomer(), addPackage(), deletePackage(), searchPackage(), createBooking(), cancelBooking(), makePayment(), showReport(), run()	Controller class: holds all data and implements the business logic and the main menu.
FileManager	file names (customers, packages, bookings, payments)	static saveAll(), loadAll(), ensureFiles()	Handles reading and writing of data files.
TravelException	message	what()	Custom exception class for validation and business-rule errors.

7. OOP Concepts Used

Concept	Where it is used
Encapsulation	Data members are private/protected and accessed through getters and member functions (e.g. seatsAvailable can only be changed through bookSeats() and releaseSeats()).
Abstraction	Person and Package are abstract classes with pure virtual functions (displayInfo(), calculatePrice()).
Inheritance	Customer inherits from Person; DomesticPackage and InternationalPackage inherit from Package.
Polymorphism	TravelAgency stores Package* pointers; calling calculatePrice() or displayDetails() runs the version of the actual derived class (runtime polymorphism via virtual functions).
Constructors / Destructors	Parameterised and default constructors initialise objects; a virtual destructor in Package ensures derived objects are deleted correctly.
Operator overloading	operator<< for Booking (printing) and operator== for Package (comparing package IDs).
Exception handling	TravelException is thrown for invalid input, insufficient seats, missing records and duplicate IDs, and caught in the menu loop.
Static members	FileManager functions and a static counter used to generate booking and payment IDs.
STL	vector for storing objects, algorithm functions (find_if, sort, transform) for searching and sorting, string and stringstream for parsing.

8. File Handling and Data Persistence

Data is stored in plain text files using `ifstream` and `ofstream`. Each record is written on one line with fields separated by the pipe character (|). Using a delimiter that is not allowed in user input keeps parsing simple and reliable.

File	Contents	Record format
customers.txt	Registered customers	id|name|phone|email|address|loyaltyPoints
packages.txt	All tour packages	type|id|destination|days|basePrice|seats|extra1|extra2
bookings.txt	All bookings	bookingId|customerId|packageId|travellers|total|status
payments.txt	Payments and refunds	paymentId|bookingId|amount|mode|status

- **Load:** At start-up *loadAll()* opens each file, reads it line by line, splits it with *stringstream* and *getline(ss, field, '|')* and rebuilds the objects. The correct derived class (Domestic/International) is created based on the first field of a package record.
- **Save:** After every successful add / update / delete / booking / payment / cancellation, *saveAll()* rewrites the files from the in-memory vectors, so the files always match the program state.
- **Error handling:** If a file is missing, it is created empty; malformed lines are skipped with a warning instead of crashing the program.
- **ID continuity:** After loading, the static ID counters are set to the highest ID found, so new IDs never clash with old ones.

9. UML Diagrams

Use Case Diagram

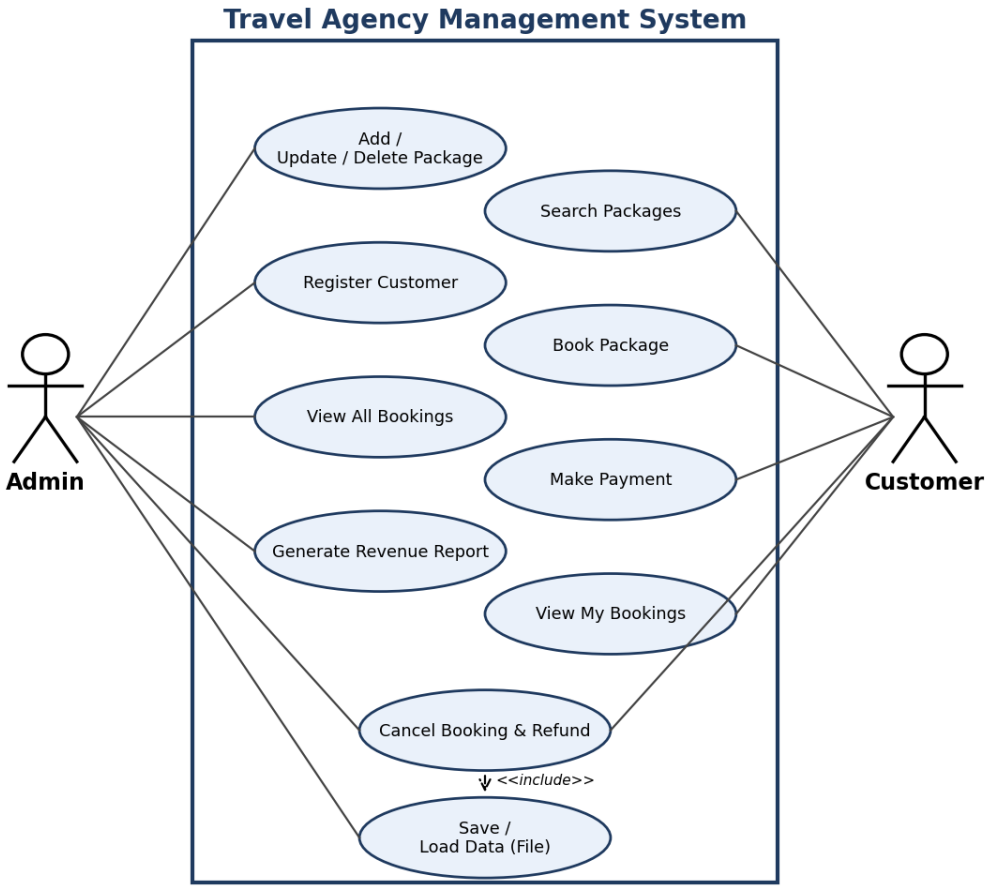
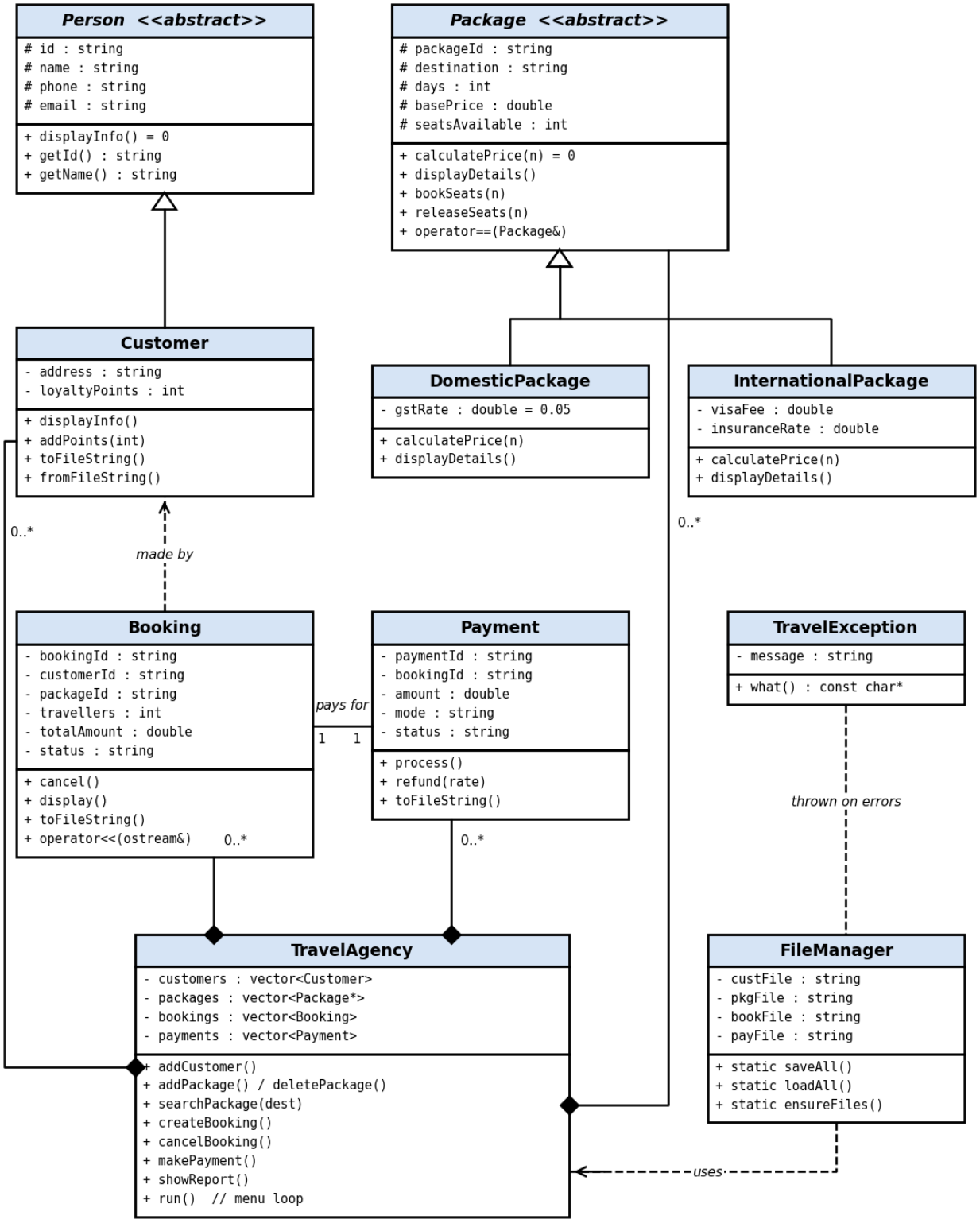


Figure 1: Use Case Diagram

The Admin manages packages, customers and reports, while the Customer searches, books, pays and cancels. Every data-changing use case includes saving to file.

Class Diagram



Legend: hollow triangle = inheritance | filled diamond = aggregation/composition
dashed line = dependency | italic class name = abstract class

Figure 2: Class Diagram

Sequence Diagram 1 - Book a Package

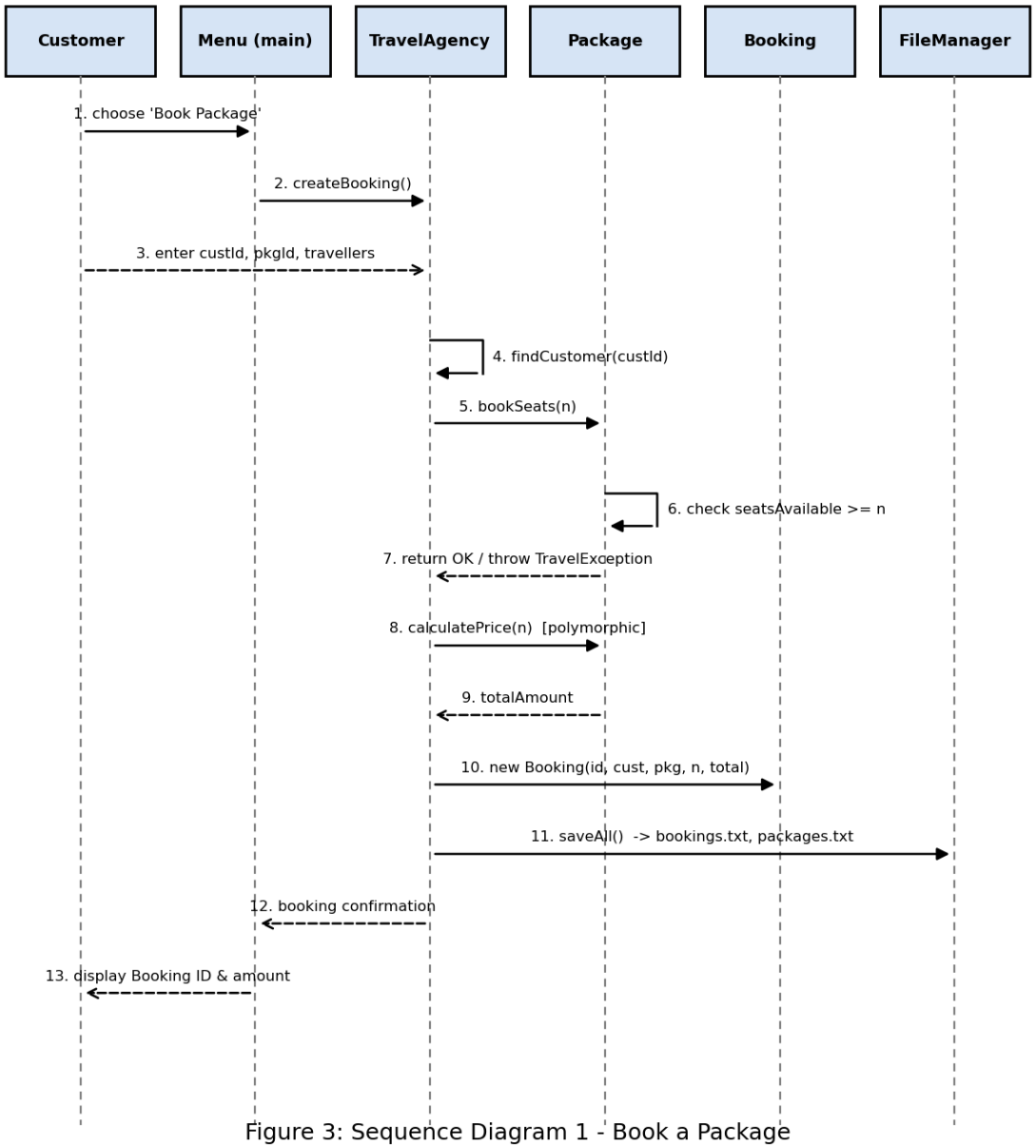


Figure 3: Sequence Diagram 1 - Book a Package

The booking flow validates the customer, reserves seats, calculates the price polymorphically, creates the *Booking* object and persists the data.

Sequence Diagram 2 - Cancel Booking and Refund

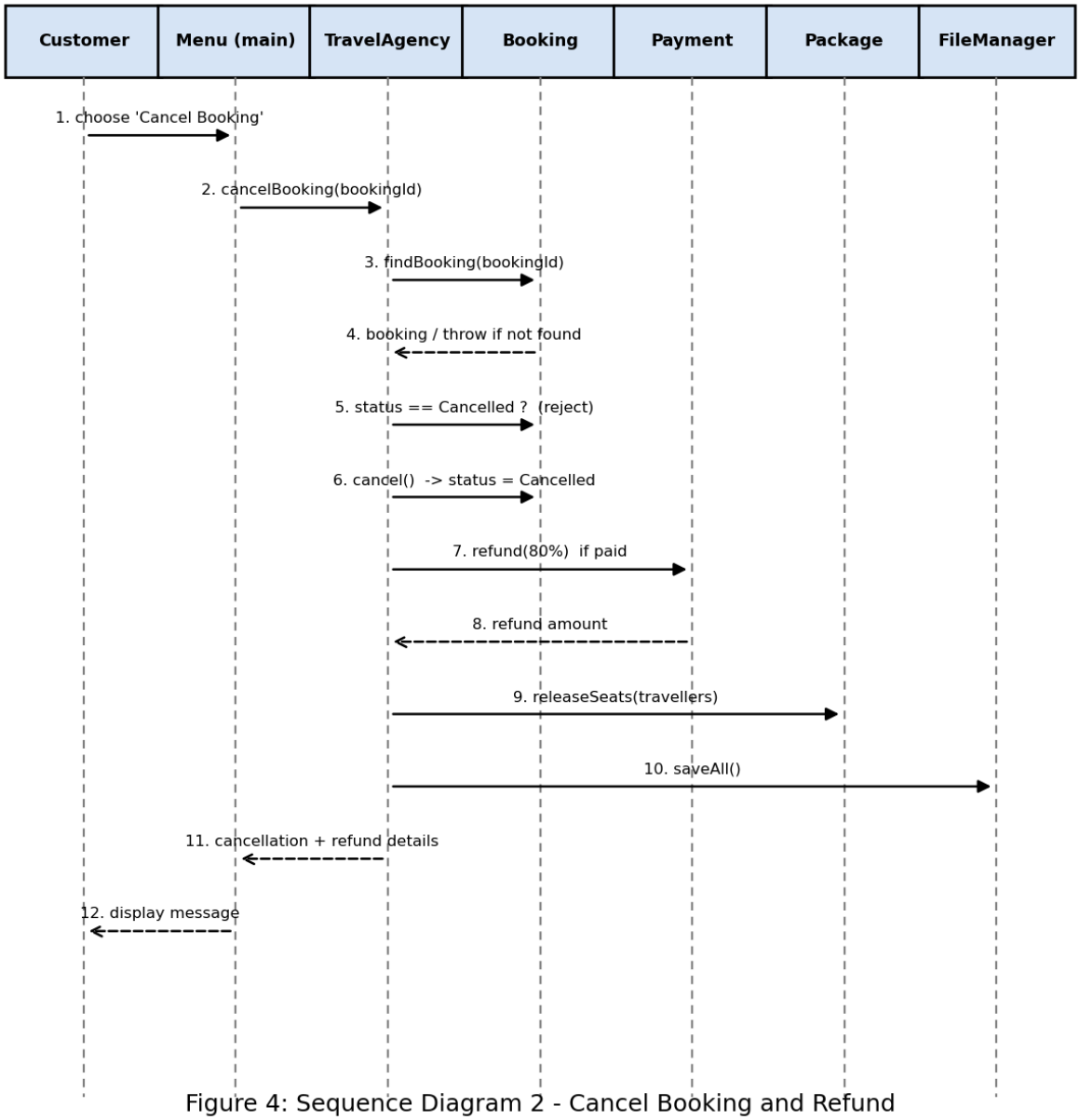


Figure 4: Sequence Diagram 2 - Cancel Booking and Refund

Cancelling a booking marks it as cancelled, refunds 80% of the paid amount, releases the seats back to the package and saves the updated data.

10. Implementation Details

The project is organised into header/source files (or a single source file for small builds) and is compiled with a C++11-or-later compiler:

```
g++ -std=c++17 -Wall -o travel_agency main.cpp
./travel_agency
```

Key design points

- **Polymorphic packages:** packages are stored as `vector<Package*>` so that domestic and international packages can be handled uniformly.
- **Price calculation:** each derived class overrides `calculatePrice(int n)`.
- **Validation:** phone must be 10 digits, e-mail must contain '@' and '.', travellers must be between 1 and 10, IDs must be unique.
- **Safety:** pointers are deleted in the TravelAgency destructor to avoid memory leaks.

Abstract base class and derived classes

```
class Package {
protected:
    string packageId, destination;
    int days, seatsAvailable;
    double basePrice;
public:
    Package(string id, string dest, int d, double price, int seats);
    virtual double calculatePrice(int n) const = 0;   // pure virtual
    virtual void displayDetails() const;
    void bookSeats(int n) {
        if (n > seatsAvailable) throw TravelException("Insufficient seats available");
        seatsAvailable -= n;
    }
    void releaseSeats(int n) { seatsAvailable += n; }
    bool operator==(const Package& p) const { return packageId == p.packageId; }
    virtual ~Package() {}
};

class DomesticPackage : public Package {
    double gstRate = 0.05;
public:
    using Package::Package;
    double calculatePrice(int n) const override { return basePrice * n * (1 + gstRate); }
};

class InternationalPackage : public Package {
    double visaFee, insuranceRate;
public:
    double calculatePrice(int n) const override {
        return (basePrice + visaFee + basePrice * insuranceRate) * n * 1.05;
    }
};
```

Booking logic

```
void TravelAgency::createBooking() {
    string cid, pid; int n;
    cout << "Customer ID: "; cin >> cid;
    cout << "Package ID : "; cin >> pid;
    cout << "Travellers : "; cin >> n;

    Customer* c = findCustomer(cid);
    if (!c) throw TravelException("Customer not found");
    Package* p = findPackage(pid);
    if (!p) throw TravelException("Package not found");
    if (n < 1 || n > 10) throw TravelException("Travellers must be between 1 and 10");

    p->bookSeats(n);                      // throws if seats are not enough
    double total = p->calculatePrice(n);   // runtime polymorphism
    bookings.push_back(Booking(nextBookingId(), cid, pid, n, total));
    FileManager::saveAll(*this);
    cout << "Booking confirmed. Total = Rs. " << total << endl;
}
```

File loading

```
void FileManager::loadBookings(TravelAgency& a) {
    ifstream in("bookings.txt");
    string line;
    while (getline(in, line)) {
        stringstream ss(line);
        string id, cid, pid, trav, total, status;
        getline(ss, id, '|');   getline(ss, cid, '|');   getline(ss, pid, '|');
        getline(ss, trav, '|'); getline(ss, total, '|'); getline(ss, status, '|');
        a.addBookingRecord(Booking(id, cid, pid, stoi(trav), stod(total), status));
    }
}
```

Main function

```
int main() {
    TravelAgency agency;
    FileManager::ensureFiles();
    FileManager::loadAll(agency);
    agency.run();                 // menu loop with try/catch around each option
    FileManager::saveAll(agency);
    return 0;
}
```

11. Testing and Results

The following test cases were executed on the console application. Sample data: package P101 (Goa, base price Rs. 12000, 30 seats), package P201 (Dubai, base Rs. 45000, visa Rs. 5000, insurance 2%), customer C001.

Test ID	Input / Operation	Expected	Actual	Status
T01	Add domestic package P101, Goa, 5 days, Rs. 12000, 30 seats	Package saved and listed in package menu	Package added and present in packages.txt	**Pass**
T02	Add package with existing ID P101	Error: duplicate ID, record rejected	'Package ID already exists' shown	**Pass**
T03	Register customer C001, valid 10-digit phone and e-mail	Customer registered	Customer saved in customers.txt	**Pass**
T04	Register customer with 9-digit phone	Validation error	'Invalid phone number' shown; not saved	**Pass**
T05	Search destination 'goa' (lower case)	P101 displayed (case-insensitive)	Goa package displayed	**Pass**
T06	Search destination 'Paris' (not available)	'No packages found' message	Message displayed	**Pass**
T07	Book P101 for C001, 2 travellers	Booking created, total Rs. 25200, seats 30 to 28	B1001 created, total 25200, seats = 28	**Pass**
T08	Book P101 for 50 travellers (only 28 seats left)	Error: insufficient seats	'Insufficient seats available'; seats unchanged	**Pass**
T09	Book with unregistered customer ID C999	Error: customer not found	'Customer not found' shown	**Pass**
T10	Pay Rs. 25200 for B1001 using UPI	Payment accepted, status Paid	Payment PY5001 recorded, status Paid	**Pass**
T11	Pay Rs. 1000 for a booking of Rs. 25200	Payment rejected (amount mismatch)	'Amount does not match booking total'	**Pass**
T12	Cancel paid booking B1001	Status Cancelled, refund Rs. 20160, seats restored to 30	Cancelled, refund 20160, seats = 30	**Pass**
T13	Cancel B1001 again	Error: already cancelled	'Booking already cancelled' shown	**Pass**
T14	Exit and restart the program	All data loaded from files	Customers, packages, bookings restored	**Pass**
T15	Delete data files and start program	New empty files created, no crash	Warning displayed; empty files created	**Pass**

Result: All 15 test cases passed. The application correctly validates input, prevents overbooking, calculates prices, handles cancellations with refunds and retains data between runs.

12. Sample Screens / Output

Main menu

```
========================================
      TRAVEL AGENCY MANAGEMENT SYSTEM
========================================
 1. Package Management
 2. Customer Management
 3. Booking
 4. Payment
 5. Reports
 0. Exit
----------------------------------------
Enter your choice: _
```

Package list / search result

```
Enter destination to search: goa

ID    Destination  Type      Days  Base Price  Seats
----  -----------  --------  ----  ----------  -----
P101  Goa          Domestic    5    12000.00     30
Total packages found: 1
```

Booking a package

```
Customer ID: C001
Package ID : P101
Travellers : 2
Booking confirmed!
Booking ID : B1001
Package    : Goa (5 days)
Travellers : 2
Total      : Rs. 25200.00 (incl. 5% GST)
Seats left : 28
```

Payment

```
Booking ID: B1001
Amount due : Rs. 25200.00
Payment mode (1.Cash 2.Card 3.UPI): 3
Enter amount: 25200
Payment successful. Payment ID: PY5001  Status: PAID
```

Cancellation and refund

```
Enter Booking ID to cancel: B1001
Booking B1001 cancelled.
Refund (80%): Rs. 20160.00
Seats restored to package P101 (now 30).
```

Error handling

```
Travellers : 50
Error: Insufficient seats available
```

Report

```
----------- AGENCY SUMMARY -----------
Total customers   : 1
Total packages    : 2
Total bookings    : 1
Cancelled bookings: 1
Revenue (net)     : Rs. 5040.00
```

13. Limitations

- The application is console-based and has no graphical user interface.
- Data is stored in plain text files, which are not encrypted and are not suitable for very large data.
- There is no login system - any user can access the admin menu.
- Only one user can use the system at a time (no multi-user or network support).
- Dates of travel are not handled and a package has a single seat pool.
- Payments are only simulated; there is no connection to a real payment gateway.
- Refund percentage is fixed (80%) and does not depend on the cancellation date.

14. Future Scope

- Add user authentication with separate Admin and Customer roles and password protection.
- Replace text files with a database such as SQLite or MySQL.
- Develop a GUI (Qt) or a web front-end for easier use.
- Add travel dates, multiple departures per package, hotel and transport booking.
- Integrate online payment gateways and send e-mail / SMS confirmations.
- Generate printable PDF tickets and invoices.
- Add discount coupons and a loyalty-point redemption system.
- Provide charts and analytics for sales and popular destinations.

15. Conclusion

The Travel Agency Management project successfully demonstrates how Object-Oriented Programming in C++ can be used to solve a real-world problem. The system manages packages, customers, bookings and payments, calculates prices through polymorphism, protects against invalid operations using exception handling, and preserves all information through file handling.

Designing the UML diagrams before coding helped in structuring the classes and understanding the flow of the application, and the 15 test cases confirmed that the program works as expected. Through this project, I gained practical experience with abstraction, inheritance, polymorphism, STL containers, file input/output and structured testing. The system can be extended further with a database, GUI and online services to become a complete travel-management solution.